Guidelines:

- Use the full train_V2.csv. Download the dataset: https://drive.google.com/file/d/1r9mtbx3bGY3v0ORQhMInrj3x4Mf9gu-L/view?usp=drive_link
- Rates are fractions; counts and IDs are exact.
- Keep unrounded values for calculations.
- Submit decimal results to six places (absolute tolerance 0.000001), unless a field is an exact count.
- Use undefined for undefined scalar statistics and null in JSON tables.
- Table rows must follow the order specified.
- JSON numeric notation is allowed in table fields.
- Enter each requested result in its labelled field.
- No file uploads or checksums are required.

E1: Return the number of player rows, distinct matches and distinct teams. A team is the composite (matchId, groupId).

Answer fields: player rows (Exact integer); matches (Exact integer); teams (Exact integer)

JSON answer format (template):

```
{ "player_rows": null,  "matches": null,  "teams": null }
```

E2: How many rows lack winPlacePerc, and how many remain for outcome analysis?

Answer fields: missing outcomes (Exact integer); labelled rows (Exact integer)

JSON answer format (template):

```
{  "missing_outcomes": null,  "labelled_rows": null }
```

E3: Save original rankPoints. Change rankPoints=-1 to missing. Change zero killPoints or winPoints to missing only when original rankPoints is not -1. Report the total player-row count and the count missing in each repaired column.

Answer fields: rows (Exact integer); missing rankPoints (Exact integer); missing killPoints (Exact integer); missing winPoints (Exact integer)

JSON answer format (template):

```
{  "rows": null,  "missing_rankpoints": null,  "missing_killpoints": null,  "missing_winpoints": null }
```

E4: Return the largest walkDistance + rideDistance + swimDistance, in metres, and its Id. Exclude rows missing any component. Break ties by ascending Id.

Answer fields: Id (Exact text, matching the question); total distance (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{  "id": null,  "total_distance": null,  "use_undefined_if_not_defined": null }
```

E5: Among labelled player rows, return the count with winPlacePerc >= 0.8, the labelled player count, and their fraction.

Answer fields: labelled players (Exact integer); top players (Exact integer); top rate (Six decimal places; use undefined if not defined)

JSON answer format (template):{
"labelled_players": null, "top_players": null, "top_rate": null, "use_undefined_if_not_defined": null
}

E6: Which of the six standard match types has the largest median walkDistance? Return the mode and median in metres. Break ties by ascending mode name.

Answer fields: matchType (Exact text, matching the question); median walkDistance (Six decimal places; use undefined if not defined)

JSON answer format (template):
{ "matchtype": null, "median_walkdistance": null, "use_undefined_if_not_defined": null }

E7: For every player with kills >= 5, calculate headshotKills / kills. Report the number of qualifying players, the mean individual share, and the number whose share equals 1. Do not submit player-level rows or a file checksum.

Answer fields: rows (Exact integer); mean headshot share (Six decimal places; use undefined if not defined); all headshot players (Exact integer)

JSON answer format (template):
{ "rows": null, "mean_headshot_share": null, "use_undefined_if_not_defined": null, "all_headshot_players": null }

E8: Return the number of teams of every observed team size, ordered by ascending team size. A team uses both matchId and groupId.

Answer fields: Answer table (JSON array of objects with exactly these keys: team_size, teams. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):
{ "answer_table": [{ "team_size": null, "teams": null }]

E9: Count rows where kills = 0 and either assists > 0 or revives > 0.

Answer fields: players (Exact integer)

JSON answer format (template):
{ "players": null }

E10: List every repeated Id and its number of appearances, ordered by ascending Id. Return [] if there are no duplicates. Do not delete records.

Answer fields: Answer table (JSON array of objects with exactly these keys: Id, appearances. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):
{ "answer_table": [{ "Id": null, "appearances": null }] }

E11: For every observed matchType, report its player-row count and its fraction of all player rows. Sort by descending count, then ascending match type.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, players, fraction. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "players": null, "fraction": null } ] }
```

E12: Define consumable use as heals + boosts. Return the Id, heals, boosts, and total for the player with the largest total. Break ties by ascending Id.

Answer fields: Id (Exact text, matching the question); heals (Exact integer); boosts (Exact integer); consumables (Exact integer)

JSON answer format (template):

```
{ "id": null, "heals": null, "boosts": null, "consumables": null }
```

E13: Among players with kills > 0, return the Id, kills, and longestKill of the player with the largest longestKill. Break ties by ascending Id.

Answer fields: Id (Exact text, matching the question); kills (Exact integer); longestKill (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "id": null, "kills": null, "longestkill": null, "use_undefined_if_not_defined": null }
```

E14: How many player rows have weaponsAcquired = 0, and what fraction of all player rows do they represent?

Answer fields: players (Exact integer); fraction (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "players": null, "fraction": null, "use_undefined_if_not_defined": null }
```

E15: Among rows with swimDistance > 0, report the player count and mean swimDistance.

Answer fields: players (Exact integer); mean swim distance (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "players": null, "mean_swim_distance": null, "use_undefined_if_not_defined": null }
```

E16: Count player rows with DBNOs > 0 and kills = 0. Also report their total DBNOs.

Answer fields: players (Exact integer); total knockdowns (Exact integer)

JSON answer format (template):

```
{ "players": null, "total_knockdowns": null }
```

E17: Among players with kills = 0 and damageDealt > 0, report the player count, total damage, and mean damage.

Answer fields: players (Exact integer); total damage (Six decimal places; use undefined if not defined); mean damage (Six decimal places; use undefined if not defined)

For total and mean damage, use float64 Pandas/NumPy aggregation, math.fsum, or Decimal from the original CSV values. Avoid naive row-by-row floating-point accumulation and rounding intermediate values. Round only final results to six decimal places (absolute tolerance 0.000001).

JSON answer format (template):

```
{ "players": null, "total_damage": null, "use_undefined_if_not_defined": null, "mean_damage": null }
```

E18: Count player rows in each of these four kill bands: 0, 1–2, 3–5, and 6+. Return the bands in this order.

Answer fields: Answer table (JSON array of objects with exactly these keys: kill_band, players. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "kill_band": null, "players": null } ] }
```

E19: For each standard mode, return the player count and the total number of healing items used, using heals only.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, players, total_heals. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "players": null, "total_heals": null } ] }
```

E20: Report the number of player rows with teamKills > 0 and the sum of teamKills across those rows.

Answer fields: players (Exact integer); total team kills (Exact integer)

JSON answer format (template):

```
{ "players": null, "total_team_kills": null }
```

E21: Among rows with roadKills > 0, return the player count, total road kills, and maximum road kills by one player.

Answer fields: players (Exact integer); total road kills (Exact integer); maximum road kills (Exact integer)

JSON answer format (template):

```
{ "players": null, "total_road_kills": null, "maximum_road_kills": null }
```

E22: How many player rows have weaponsAcquired >= 10, and what fraction of all player rows do they represent?

Answer fields: players (Exact integer); fraction (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "players": null, "fraction": null, "use_undefined_if_not_defined": null }
```

E23: Return the 25th percentile, median, 75th percentile, and interquartile range of walkDistance across all player rows, using linear interpolation.

Answer fields: q1 (Six decimal places; use undefined if not defined); median (Six decimal places; use undefined if not defined); q3 (Six decimal places; use undefined if not defined); iqr (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "q1": null, "use_undefined_if_not_defined": null, "median": null, "q3": null, "iqr": null }
```

E24: Count player rows with both heals > 0 and boosts > 0, and report their fraction of all player rows.

Answer fields: players (Exact integer); fraction (Six decimal places; use undefined if not defined)

JSON answer format (template):
{ "players": null, "fraction": null, "use_undefined_if_not_defined": null }

E25: Among player rows with rideDistance > 0 and walkDistance = 0, return the player count and median rideDistance.

Answer fields: players (Exact integer); median ride distance (Six decimal places; use undefined if not defined)

JSON answer format (template):

{ "players": null, "median_ride_distance": null, "use_undefined_if_not_defined": null }

M1: Compare player-weighted and team-weighted mean winPlacePerc on the same eligible teams. Eligible teams have exactly one distinct nonmissing outcome; exclude entirely unlabelled or contradictory teams. For the player mean retain labelled rows only. Report both means, eligible team count and excluded team count.

Answer fields: player mean (Six decimal places; use undefined if not defined); team mean (Six decimal places; use undefined if not defined); eligible teams (Exact integer); excluded teams (Exact integer)

JSON answer format (template):
{ "player_mean": null, "use_undefined_if_not_defined": null, "team_mean": null, "eligible_teams": null, "excluded_teams": null }

M2: For every team, calculate the sum of squared individual kill shares: sum((kills_i / total_team_kills)^2). Mark zero-kill teams as undefined. Report total teams (rows), undefined team count, and the mean concentration across defined teams, weighting each team equally.

Answer fields: rows (Exact integer); undefined teams (Exact integer); mean concentration (Six decimal places; use undefined if not defined)

JSON answer format (template):
{ "rows": null, "undefined_teams": null, "mean_concentration": null, "use_undefined_if_not_defined": null }

M3: For every player, convert damageDealt into a within-match percentile using ascending average rank: (rank - 1)/(n - 1). Set singleton matches to 0.5. Report the total player count (rows), the mean percentile across all players, and the number of players with percentile >= 0.9.

Answer fields: rows (Exact integer); mean damage percentile (Six decimal places; use undefined if not defined); players at or above 90th (Exact integer)

JSON answer format (template):
{ "rows": null, "mean_damage_percentile": null, "use_undefined_if_not_defined": null, "players_at_or_above_90th": null }

M4: List teams containing more than one distinct nonmissing winPlacePerc. Return matchId, groupId, distinct_outcomes in ascending composite-key order, or [] if none.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchId, groupId, distinct_outcomes. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):
{ "answer_table": [{ "matchId": null, "groupId": null, "distinct_outcomes": null }] }

M5: Within squad-fpp, retain teams with exactly one distinct nonmissing outcome. Sum walking distance across all observed team members using exact decimal arithmetic. The source values have at most four decimal places, so an equivalent method is to convert walkDistance to integer units of 0.0001 metres before summing and comparing. Compare team top-placement rates above the median team walk distance (> median) and below-or-equal (<= median). Report the median in metres, team counts, top-team counts and rates; return below_or_equal then above.

Answer fields: Answer table (JSON array of objects with exactly these keys: walk_group, median_team_walk, teams, top_teams, top_rate. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):
{ "answer_table": [{ "walk_group": null, "median_team_walk": null, "teams": null, "top_teams": null, "top_rate": null }] }

M6: Return the number of winning player rows (winPlacePerc = 1) and winning composite-key teams. For this question retain all matches, including malformed or unusual matches; report malformed_matches_excluded as no.

Answer fields: winning players (Exact integer); winning teams (Exact integer); malformed matches excluded (Exact text, matching the question)

JSON answer format (template):
{ "winning_players": null, "winning_teams": null, "malformed_matches_excluded": null }

M7: Sort all distinct match IDs ascending. Assign every fifth ID (positions 5, 10, 15, ... using one-based positions) to validation, all others to train. Report total match count (rows), match counts and player counts per split. Complete matches must remain together.

Answer fields: rows (Exact integer); train matches (Exact integer); validation matches (Exact integer); train players (Exact integer); validation players (Exact integer)

JSON answer format (template):
{ "rows": null, "train_matches": null, "validation_matches": null, "train_players": null, "validation_players": null }

M8: For each match, compare the observed number of distinct groupId values with its reported numGroups. Report total matches, matches with more than one distinct reported numGroups value, comparable matches, matching matches, and mismatching matches. A match is comparable only if its reported value is constant across all rows.

Answer fields: matches (Exact integer); inconsistent matches (Exact integer); comparable matches (Exact integer); matching matches (Exact integer); mismatching matches (Exact integer)

JSON answer format (template):

{ "matches": null, "inconsistent_matches": null, "comparable_matches": null, "matching_matches": null, "mismatching_matches": null }

M9: In squad-fpp, keep teams with at least two observed players and positive total damageDealt. For every remaining player, divide their damage by their team's total damage. Return the five players with the largest shares, including matchId, groupId, Id, individual damage, team damage, and share. Break ties by ascending matchId, then groupId, then Id.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchId, groupId, Id, damageDealt, team_damage, damage_share. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "matchId": null, "groupId": null, "Id": null, "damageDealt": null, "team_damage": null, "damage_share": null }] }

M10: For labelled player rows, define low placement as 0 <= winPlacePerc < 0.2, middle as 0.2 <= winPlacePerc < 0.8, and top as 0.8 <= winPlacePerc <= 1. Report player counts and mean heals + boosts in the order low, middle, top.

Answer fields: Answer table (JSON array of objects with exactly these keys: placement_band, players, mean_consumables. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "placement_band": null, "players": null, "mean_consumables": null }] }

M11: In each standard mode, keep players with kills >= 5. Report the number of qualifying players, the pooled headshot share sum(headshotKills) / sum(kills), and the arithmetic mean of individual headshotKills / kills shares.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, players, pooled_headshot_share, mean_player_headshot_share. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "matchType": null, "players": null, "pooled_headshot_share": null, "mean_player_headshot_share": null }] }

M12: For each standard-mode match, select one kill leader by descending kills, then descending damageDealt, then ascending Id. Select leaders before dropping missing outcomes. For each standard mode, report the total number of selected leaders, the number with labelled outcomes, their mean placement, and their top placement rate.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, leaders, labelled_leaders, mean_placement, top_rate. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "matchType": null, "leaders": null, "labelled_leaders": null, "mean_placement": null, "top_rate": null }] }

M13: For labelled players in each standard mode, report the player count and Spearman correlation between walkDistance and winPlacePerc. Use average ranks for ties. Return an undefined correlation if either variable is constant or fewer than two labelled rows remain.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, players, spearman. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "players": null, "spearman": null } ] }
```

M14: In squad-fpp, keep teams with exactly four observed players and exactly one distinct nonmissing winPlacePerc value. A team may have some missing outcomes if all observed outcomes agree. Compare teams whose summed revives is zero with teams whose summed revives is positive. For each group, report team count, top-placement team count, and top placement rate. Return the zero group first.

Answer fields: Answer table (JSON array of objects with exactly these keys: revive_group, teams, top_teams, top_rate. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "revive_group": null, "teams": null, "top_teams": null, "top_rate": null } ] }
```

M15: Among labelled rows in the six standard modes, compare the pooled player mean winPlacePerc with the unweighted mean of the six mode-specific means. Return both and equal_mode_mean - pooled_player_mean. If any standard mode has no labelled rows, the equal-mode mean and difference are undefined.

Answer fields: pooled player mean (Six decimal places; use undefined if not defined); equal mode mean (Six decimal places; use undefined if not defined); difference (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "pooled_player_mean": null, "use_undefined_if_not_defined": null, "equal_mode_mean": null, "difference": null }
```

M16: For every player in a team with at least two observed players, calculate mean teammate kills as (team_kills - own_kills) / (team_size - 1). Define the kill gap as own_kills - mean_teammate_kills. Return the five largest gaps, with the composite team key, player Id, own kills, mean teammate kills, and gap. Break ties by ascending matchId, groupId, then Id.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchId, groupId, Id, kills, mean_teammate_kills, kill_gap. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchId": null, "groupId": null, "Id": null, "kills": null, "mean_teammate_kills": null, "kill_gap": null } ] }
```

M17: Take the first match in ascending matchId order. Calculate each player's damage z-score as (damageDealt - match_mean) / match_std, using the population standard deviation (ddof=0). If the

standard deviation is zero, assign every player a z-score of zero. Return the first five players by ascending Id (or all if fewer than five), including match and player IDs, damage, match mean, match standard deviation, and z-score.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchId, Id, damageDealt, match_mean, match_std, damage_zscore. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchId": null, "Id": null, "damageDealt": null, "match_mean": null, "match_std": null, "damage_zscore": null } ] }
```

M18: Keep labelled player rows with maxPlace > 1. For each, calculate the distance from winPlacePerc to its nearest legal value in {0, 1/(maxPlace-1), ..., 1}. Report eligible player count, the number whose distance is strictly greater than 0.000050001, and the largest distance. The tolerance allows four-decimal target rounding.

Answer fields: eligible players (Exact integer); outside tolerance (Exact integer); maximum grid distance (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "eligible_players": null, "outside_tolerance": null, "maximum_grid_distance": null, "use_undefined_if_not_defined": null }
```

M19: Keep matches where both matchType and matchDuration are constant across their player rows, then retain the six standard modes. For each mode, report match count and mean and median matchDuration. Count each match once.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, matches, mean_duration, median_duration. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "matches": null, "mean_duration": null, "median_duration": null } ] }
```

M20: For each match across all modes, select a kill leader by descending kills, then ascending Id; independently select a damage leader by descending damageDealt, then ascending Id. Report total matches, matches where the two selected IDs agree, matches where they differ, and the fraction where they agree.

Answer fields: matches (Exact integer); same leader (Exact integer); different leader (Exact integer); same fraction (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "matches": null, "same_leader": null, "different_leader": null, "same_fraction": null, "use_undefined_if_not_defined": null }
```

M21: Within each standard mode, calculate Q1 and Q3 of walkDistance using linear interpolation. Set an upper fence of Q3 + 1.5*(Q3-Q1). Report Q1, Q3, the fence, player count, count strictly above the fence, and fraction strictly above it.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, q1, q3, upper_fence, players, above_fence, above_fraction. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "q1": null, "q3": null, "upper_fence": null, "players": null, "above_fence": null, "above_fraction": null } ] }
```

M22: Use labelled rows in the six standard modes. In each mode, compare mean placement for players with boosts >= 1 against players with boosts = 0. Report both group counts and means and the difference boosted_mean - zero_boost_mean. Then report that same difference with all six modes pooled, and the unweighted mean of the six within-mode differences. If any mode lacks either group, the equal-mode difference is undefined. Return the six modes alphabetically, then ALL_PLAYERS, then EQUAL_MODES; group counts and group means are undefined in the last row.

Answer fields: Answer table (JSON array of objects with exactly these keys: matchType, zero_players, boosted_players, zero_mean, boosted_mean, difference. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

```
{ "answer_table": [ { "matchType": null, "zero_players": null, "boosted_players": null, "zero_mean": null, "boosted_mean": null, "difference": null } ] }
```

M23: In squad-fpp, keep teams of exactly four observed players. Report team count and zero-kill team count. Among positive-kill teams, compare the mean of team assists/kills with sum(team assists)/sum(team kills). Units are assists per kill; zero-kill teams are excluded from both ratios.

Answer fields: teams (Exact integer); zero kill teams (Exact integer); mean assists per kill (Six decimal places; use undefined if not defined); pooled assists per kill (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "teams": null, "zero_kill_teams": null, "mean_assists_per_kill": null, "use_undefined_if_not_defined": null, "pooled_assists_per_kill": null }
```

M24: For labelled solo-fpp players, define a positive outcome as winPlacePerc >= 0.8. Use damageDealt as a score. Calculate ROC AUC as the probability that a random positive has greater damage than a random negative, with half credit for tied damage. Report positive and negative player counts and AUC. If either class is absent, AUC is undefined.

Answer fields: positive players (Exact integer); negative players (Exact integer); damage auc (Six decimal places; use undefined if not defined)

JSON answer format (template):

```
{ "positive_players": null, "negative_players": null, "damage_auc": null, "use_undefined_if_not_defined": null }
```

M25: In duo-fpp, retain exactly two-player teams with exactly one distinct nonmissing outcome. Separate equal-kill teams from unequal-kill teams. For each group report team count, mean absolute

teammate kill gap, mean placement and top-placement rate. Return equal then unequal. Partly missing but consistent outcomes are allowed.

Answer fields: Answer table (JSON array of objects with exactly these keys: kill_balance, teams, mean_kill_gap, mean_placement, top_rate. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "kill_balance": null, "teams": null, "mean_kill_gap": null, "mean_placement": null, "top_rate": null }] }

H1: Use the first match in ascending matchId order. Sum walkDistance per groupId and calculate ascending average rank(pct=True) within that complete match. Keep the four lowest-scoring teams, ties by ascending groupId. Assume maxPlace=6 and legal values {0,0.2,0.4,0.6,0.8,1}. Use dynamic programming to assign distinct legal values in the selected score/groupId order, increasing in that order, minimising squared error. Unused values are allowed. If costs tie, select the lexicographically smallest sequence of legal values. Return four rows containing matchId, groupId, predicted_score, assigned_value and minimum_squared_error (the total, repeated per row).

Answer fields: Answer table (JSON array of objects with exactly these keys: matchId, groupId, predicted_score, assigned_value, minimum_squared_error. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "matchId": null, "groupId": null, "predicted_score": null, "assigned_value": null, "minimum_squared_error": null }] }

H2: Take every 50th distinct matchId in ascending order, starting with the first. Keep labelled players. Predictor A is always 0.5; B is ascending average rank(pct=True) of walkDistance within each remaining match. Report match count, player count, both match-weighted MAEs, both pooled player MAEs, the better predictor for each weighting (A, B or tie), and whether the ranking changes (yes/no). Lower MAE is better.

Answer fields: matches (Exact integer); players (Exact integer); match mae A (Six decimal places; use undefined if not defined); match mae B (Six decimal places; use undefined if not defined); player mae A (Six decimal places; use undefined if not defined); player mae B (Six decimal places; use undefined if not defined); match winner (Exact text, matching the question); player winner (Exact text, matching the question); ranking changes (Exact text, matching the question)

JSON answer format (template):

{ "matches": null, "players": null, "match_mae_a": null, "use_undefined_if_not_defined": null, "match_mae_b": null, "player_mae_a": null, "player_mae_b": null, "match_winner": null, "player_winner": null, "ranking_changes": null }

H3: A team splits 100 points proportionally to score = 2*kills* + *assists* + *3*revives. If all scores are zero, split equally. Take the first match containing an exactly four-player team, then the smallest groupId among its four-player teams. Select the lowest baseline allocation, ties by ascending Id. Add 0,1,2,3 revives to this player only and recompute all allocations each time. Report matchId, groupId,

Id, all four selected-player allocations and their maximum. Use fractional points without intermediate rounding.

Answer fields: matchId (Exact text, matching the question); groupId (Exact text, matching the question); Id (Exact text, matching the question); extra 0 allocation (Six decimal places; use undefined if not defined); extra 1 allocation (Six decimal places; use undefined if not defined); extra 2 allocation (Six decimal places; use undefined if not defined); extra 3 allocation (Six decimal places; use undefined if not defined); maximum allocation (Six decimal places; use undefined if not defined)

JSON answer format (template):

{ "matchid": null, "groupid": null, "id": null, "extra_0_allocation": null, "use_undefined_if_not_defined": null, "extra_1_allocation": null, "extra_2_allocation": null, "extra_3_allocation": null, "maximum_allocation": null }

H4: Use labelled squad-fpp players. A match is eligible when it contains at least one player with revives=0 and one with revives>0. Take the first 12 eligible matches ascending by matchId. Within each, compute mean placement for revives>0 minus mean placement for revives=0. The statistic is the unweighted mean of these 12 differences. Enumerate all 4096 independent +/- sign assignments to the differences. Count absolute permuted statistics >= abs(observed)-1e-12. Return match count, observed statistic, extreme count, total patterns and the two-sided exact p-value. This is a sign-symmetry test, not evidence of a causal revive effect.

Answer fields: matches (Exact integer); observed mean difference (Six decimal places; use undefined if not defined); extreme sign patterns (Exact integer); total sign patterns (Exact integer); two sided p value (Six decimal places; use undefined if not defined)

JSON answer format (template):

{ "matches": null, "observed_mean_difference": null, "use_undefined_if_not_defined": null, "extreme_sign_patterns": null, "total_sign_patterns": null, "two_sided_p_value": null }

H5: Sort all distinct matchId values in ascending order, including matches with missing target values. Assign every fifth match (positions 5, 10, 15, ...) to the validation set, and all remaining matches to the training set. Remove rows with missing winPlacePerc after splitting.

Use the following features in this order:

1. log1p(walkDistance)
2. log1p(boosts)
3. log1p(kills)

Standardise the features using the training-set means and population standard deviations. Replace any zero standard deviation with 1. Do not standardise using validation data.

Set the intercept to the mean winPlacePerc of the training set. Fit the ridge regression coefficients using:

$$\beta = (Z_{train}^{T} Z_{train} + 10I)^{-1} Z_{train}^{T} (y_{train} - intercept)$$

Do not penalise the intercept.

Clip validation predictions to the range [0, 1].Compare the ridge model against a baseline that predicts the training-set mean placement for every validation player. Calculate the **match-weighted validation mean absolute error (MAE)** for both predictors.

Report the following:

- Training player count
- Validation player count
- Intercept
- Three ridge coefficients, in the specified feature order
- Ridge model match-weighted validation MAE
- Baseline match-weighted validation MAE
- Better predictor (ridge, baseline or tie)

Answer fields: train players (Exact integer); validation players (Exact integer); intercept (Six decimal places; use undefined if not defined); beta walkDistance (Six decimal places; use undefined if not defined); beta boosts (Six decimal places; use undefined if not defined); beta kills (Six decimal places; use undefined if not defined); ridge match mae (Six decimal places; use undefined if not defined); baseline match mae (Six decimal places; use undefined if not defined); better predictor (Exact text, matching the question)

JSON answer format (template):
{ "train_players": null, "validation_players": null, "intercept": null, "use_undefined_if_not_defined": null, "beta_walkdistance": null, "beta_boosts": null, "beta_kills": null, "ridge_match_mae": null, "baseline_match_mae": null, "better_predictor": null }

H6: Exclude entirely unlabelled or contradictory teams, then keep labelled rows. Compare two splits: (1) sort eligible player rows by Id and hold out positions 5,10,15,...; (2) hold out every fifth match from the sorted complete original match list, as in M7. For each split, predict a validation player using the single consistent outcome of a training teammate when one exists; otherwise use that split's global training mean placement. Report split (player then match), train and validation row counts, validation rows with a training teammate, their fraction, and pooled validation MAE. Count teammates by both matchId and groupId. The low error of leaked labels is not a deployable predictive model.

Answer fields: Answer table (JSON array of objects with exactly these keys: split, train_players, validation_players, validation_with_training_teammate, teammate_leakage_rate, player_mae. Follow the specified row order. Use [] for no rows and null for undefined statistics.)

JSON answer format (template):

{ "answer_table": [{ "split": null, "train_players": null, "validation_players": null, "validation_with_training_teammate": null, "teammate_leakage_rate": null, "player_mae": null }] }